Request Mapping in Spring Boot

Request Mapping is used to connect a **URL request from the client** to a **Java method in a Controller**.

In simple words:

Request Mapping tells Spring which method should execute when a particular URL is requested.

For example:

```
Browser
   |
   | GET /hello
   ↓
Spring Boot Controller
   |
   ↓
hello() method
```

---

1. Simple Example

Create a controller:

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Student!";
    }
}
```

Now run the Spring Boot application and open:

```
http://localhost:8080/hello
```

Output

```
Hello Student!
```

Here:

```
@GetMapping("/hello")
```

means:

When a **GET request** comes to `/hello`, execute the `hello()` method.

---

2. What is `@RequestMapping`?

`@RequestMapping` is a general-purpose annotation for mapping HTTP requests.

Example:

```
@RestController
public class StudentController {

    @RequestMapping("/student")
    public String student() {
        return "Student Information";
    }
}
```

When you visit:

```
http://localhost:8080/student
```

Output:

```
Student Information
```

---

3. Different HTTP Request Mappings

Spring provides specialized annotations that are easier to use:

Annotation	HTTP Method	Purpose
`@GetMapping`	GET	Retrieve data
`@PostMapping`	POST	Create/save data
`@PutMapping`	PUT	Update data
`@DeleteMapping`	DELETE	Delete data
`@PatchMapping`	PATCH	Partially update data
`@RequestMapping`	Any/specified	General mapping

For example:

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping
    public String getStudents() {
        return "Get Students";
    }

    @PostMapping
    public String addStudent() {
        return "Student Added";
    }

    @PutMapping
    public String updateStudent() {
        return "Student Updated";
```

```
    }

    @DeleteMapping
    public String deleteStudent() {
        return "Student Deleted";
    }
}
```

The URLs are:

```
GET     /students
POST    /students
PUT     /students
DELETE  /students
```

---

4. Class-Level and Method-Level Mapping

You can put `@RequestMapping` at the **class level**.

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/all")
    public String getAllStudents() {
        return "All Students";
    }

    @GetMapping("/active")
    public String getActiveStudents() {
        return "Active Students";
    }
}
```

Now:

```
GET /students/all
```

calls:

```
getAllStudents()
```

And:

```
GET /students/active
```

calls:

```
getActiveStudents()
```

The mapping is combined:

```
Class Mapping
      +
Method Mapping
      =
Final URL
```

For example:

```
/students + /all
       ↓
/students/all
```

---

5. `@RequestMapping` with HTTP Method

You can also specify the HTTP method directly:

```
@RequestMapping(
    value = "/students",
    method = RequestMethod.GET
)
public String getStudents() {
    return "Student List";
}
```

You need:

```
import org.springframework.web.bind.annotation.RequestMethod;
```

However, this is usually written more simply as:

```
@GetMapping("/students")
public String getStudents() {
    return "Student List";
}
```

So `@GetMapping` is basically a convenient way to express a GET request mapping.

Simple Example to Remember

```
@RestController
@RequestMapping("/student")
public class StudentController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Student";
    }
}
```

Request:

```
GET http://localhost:8080/student/hello
```

Flow:

```
Client
  |
  | GET /student/hello
  ↓
StudentController
  |
  | @RequestMapping("/student")
  | @GetMapping("/hello")
  ↓
hello()
```

```
  |
  ↓
"Hello Student"
```

In one sentence

Request Mapping is the mechanism in Spring MVC that maps an HTTP request URL and HTTP method to a controller method.

`@RequestMapping` can be used with **both `@Controller` and `@RestController`**.

1. With `@Controller`

```
@Controller
@RequestMapping("/student")
public class StudentController {

    @RequestMapping("/home")
    public String home() {
        return "student";
    }
}
```

Here, `student` is usually the name of a **view/template** such as `student.html`.

```
GET /student/home
        ↓
home()
        ↓
student.html
```

2. With `@RestController`

```
@RestController
@RequestMapping("/student")
public class StudentController {

    @RequestMapping("/home")
    public String home() {
        return "Student Information";
    }
}
```

Here, the return value is sent directly as the **HTTP response**.

```
GET /student/home
        ↓
home()
        ↓
"Student Information"
```

Key difference

	`@Controller`	`@RestController`
`@RequestMapping`	☑ Yes	☑ Yes
Usually used for	Web pages/views	REST APIs
Return value	View name	Response data
Equivalent behavior	`@Controller`	`@Controller` + `@ResponseBody`

So, remember:

`@RequestMapping` is not specific to `@RestController`. It can be used with both `@Controller` and `@RestController`.

What is Response Entity? Explain with small example code.

What is `ResponseEntity` in Spring Boot?

`ResponseEntity` is a Spring class used to **control the complete HTTP response** returned by a REST API.

It allows you to specify:

- **Response body** — the actual data/message
- **HTTP status code** — `200`, `404`, `201`, etc.
- **HTTP headers** — optional additional response information

In simple words:

`ResponseEntity` gives you control over what your API sends back to the client.

---

1. Without `ResponseEntity`

```
@RestController
public class StudentController {

    @GetMapping("/student")
    public String getStudent() {
        return "Student Found";
    }
}
```

Response:

```
Student Found
```

Spring normally returns HTTP status:

```
200 OK
```

But sometimes we want to control the status code.

---

2. Using `ResponseEntity`

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StudentController {

    @GetMapping("/student")
    public ResponseEntity<String> getStudent() {

        return ResponseEntity.ok("Student Found");
    }
}
```

When you request:

```
GET http://localhost:8080/student
```

Response:

```
Status: 200 OK

Body:
Student Found
```

---

3. Returning `404 Not Found`

This is where `ResponseEntity` becomes very useful.

```
@GetMapping("/student")
public ResponseEntity<String> getStudent() {

    boolean studentExists = false;

    if (studentExists) {
        return ResponseEntity.ok("Student Found");
    }

    return ResponseEntity.notFound().build();
}
```

If the student does not exist:

```
HTTP Status: 404 Not Found
```

---

4. Returning Different Status Codes

Some commonly used methods are:

```
ResponseEntity.ok()

→ 200 OK

ResponseEntity.notFound()

→ 404 Not Found
```

```
ResponseEntity.badRequest()

→ 400 Bad Request

ResponseEntity.status(201)

→ 201 Created
```

For example:

```
@PostMapping("/student")
public ResponseEntity<String> saveStudent() {

    return ResponseEntity
            .status(201)
            .body("Student Created");
}
```

Response:

```
HTTP Status: 201 Created

Body:
Student Created
```

---

5. `ResponseEntity` with an Object

Suppose we have:

```
public class Student {

    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}
```

Controller:

```
@RestController
public class StudentController {

    @GetMapping("/student")
    public ResponseEntity<Student> getStudent() {

        Student student = new Student(101, "Rahul");

        return ResponseEntity.ok(student);
    }
}
```

The API response will be JSON:

```
{
    "id": 101,
    "name": "Rahul"
}
```

with:

```
HTTP Status: 200 OK
```

Simple Flow

```
Client
   |
   | GET /student
   ↓
Controller
   |
   | ResponseEntity
   ↓
+-----------------------+
| Status: 200 OK        |
| Body: Student object  |
| Headers: Optional     |
+-----------------------+
   |
   ↓
Client
```

Remember

`ResponseEntity` = Response Body + HTTP Status + HTTP Headers

For example:

```
return ResponseEntity.ok(student);
```

means:

```
Status → 200 OK
Body   → student
```

And:

```
return ResponseEntity.notFound().build();
```

means:

```
Status → 404 Not Found
Body   → empty
```

For REST APIs, `ResponseEntity` is especially useful when your API needs to return **different HTTP status codes depending on the result**.

Write a complete example for Response Entity

Sure. Here is a **complete but very simple Spring Boot example** using `ResponseEntity` for a Student REST API.

We will create:

- `Student.java` → Model
- `StudentController.java` → REST Controller
- `StudentApplication.java` → Main class

The API will demonstrate:

- `200 OK`
- `201 CREATED`
- `404 NOT FOUND`
- `400 BAD REQUEST`

---

1. Project Structure

```
response-entity-demo/
│
├── src/
│   └── main/
│       └── java/
│           └── com/
│               └── example/
│                   └── demo/
│                       ├── StudentApplication.java
│                       ├── Student.java
│                       └── StudentController.java
│
└── pom.xml
```

---

2. **StudentApplication.java**

This is the main Spring Boot class.

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApplication {

    public static void main(String[] args) {

        SpringApplication.run(StudentApplication.class, args);

    }
}
```

---

3. **Student.java**

This is our Student model.

```
package com.example.demo;

public class Student {

    private int id;
    private String name;
    private String course;

    public Student() {
    }

    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

---

4. **StudentController.java**

Now we use `ResponseEntity`.

```
package com.example.demo;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {

    private List<Student> students = new ArrayList<>();
```

```
    // GET all students
    @GetMapping
    public ResponseEntity<List<Student>> getAllStudents() {

        return ResponseEntity.ok(students);
    }


    // GET student by ID
    @GetMapping("/{id}")
    public ResponseEntity<?> getStudentById(@PathVariable int id) {

        for (Student student : students) {

            if (student.getId() == id) {

                return ResponseEntity.ok(student);
            }
        }

        return ResponseEntity
                .notFound()
                .build();
    }


    // POST - Add student
    @PostMapping
    public ResponseEntity<?> addStudent(@RequestBody Student student) {

        if (student.getName() == null ||
            student.getName().isEmpty()) {

            return ResponseEntity
                    .badRequest()
                    .body("Student name is required");
        }

        students.add(student);

        return ResponseEntity
                .status(201)
                .body(student);
    }


    // DELETE - Delete student
    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteStudent(@PathVariable int id) {

        for (Student student : students) {

            if (student.getId() == id) {

                students.remove(student);

                return ResponseEntity
                        .ok("Student deleted successfully");
            }
        }

        return ResponseEntity
                .notFound()
                .build();
    }
}
```

---

5. How `ResponseEntity` Works Here

Look at this:

```
return ResponseEntity.ok(student);
```

It means:

```
HTTP Status → 200 OK
Response Body → student
```

For a student that doesn't exist:

```
return ResponseEntity.notFound().build();
```

It means:

```
HTTP Status → 404 NOT FOUND
Response Body → Empty
```

For invalid input:

```
return ResponseEntity
        .badRequest()
        .body("Student name is required");
```

It means:

```
HTTP Status → 400 BAD REQUEST
Response Body → Student name is required
```

For successfully creating a student:

```
return ResponseEntity
        .status(201)
        .body(student);
```

It means:

```
HTTP Status → 201 CREATED
Response Body → Student object
```

---

6. Run the Application

From your project directory:

```
mvn spring-boot:run
```

You should see something similar to:

```
Tomcat started on port 8080
Started StudentApplication
```

Your application is now running at:

```
http://localhost:8080
```

---

7. Test GET All Students

Open:

```
http://localhost:8080/students
```

Initially, there are no students.

Response:

```
[]
```

Status:

```
200 OK
```

---

8. Add a Student

Use Postman, Thunder Client, or another REST client.

Select:

```
POST
```

URL:

```
http://localhost:8080/students
```

Select:

```
Body → raw → JSON
```

Send:

```
{
    "id": 101,
    "name": "Rahul",
    "course": "Java"
}
```

Response:

```
{
    "id": 101,
    "name": "Rahul",
    "course": "Java"
}
```

Status:

```
201 Created
```

---

9. Get Student by ID

Send:

```
GET http://localhost:8080/students/101
```

Response:

```
{
    "id": 101,
    "name": "Rahul",
    "course": "Java"
}
```

Status:

```
200 OK
```

Because this code executes:

```
return ResponseEntity.ok(student);
```

---

10. Search for a Student That Doesn't Exist

Send:

```
GET http://localhost:8080/students/999
```

There is no student with ID `999`.

Therefore:

```
return ResponseEntity
        .notFound()
        .build();
```

Response:

```
404 Not Found
```

This is one of the major advantages of `ResponseEntity`.

---

11. Test Bad Request

Try adding a student without a name.

```
POST http://localhost:8080/students
```

JSON:

```
{
    "id": 102,
    "name": "",
    "course": "Spring Boot"
}
```

The following condition becomes true:

```
if (student.getName() == null ||
    student.getName().isEmpty()) {
```

So the controller returns:

```
return ResponseEntity
        .badRequest()
        .body("Student name is required");
```

Response:

```
400 Bad Request
```

Body:

```
Student name is required
```

---

12. Delete a Student

Send:

```
DELETE http://localhost:8080/students/101
```

Response:

```
200 OK
```

Body:

```
Student deleted successfully
```

If you try to delete it again:

```
DELETE http://localhost:8080/students/101
```

You will get:

```
404 Not Found
```

Complete API Summary

HTTP Method	URL	Purpose	Response
GET	`/students`	Get all students	`200 OK`
GET	`/students/101`	Get student	`200 OK`
GET	`/students/999`	Student not found	`404 NOT FOUND`

HTTP Method	URL	Purpose	Response
POST	`/students`	Add student	`201 CREATED`
POST	`/students`	Invalid student	`400 BAD REQUEST`
DELETE	`/students/101`	Delete student	`200 OK`
DELETE	`/students/999`	Student not found	`404 NOT FOUND`

Most Important Concept

Think of this:

```
ResponseEntity.ok(student);
```

as:

```
+-----------------------------+
|       HTTP RESPONSE         |
+-----------------------------+
| Status : 200 OK             |
| Body   : Student Object     |
| Headers: Optional           |
+-----------------------------+
```

While:

```
ResponseEntity
    .status(201)
    .body(student);
```

means:

```
+-----------------------------+
|       HTTP RESPONSE         |
+-----------------------------+
| Status : 201 CREATED        |
| Body   : Student Object     |
| Headers: Optional           |
+-----------------------------+
```

And:

```
ResponseEntity.notFound().build();
```

means:

```
+-----------------------------+
|       HTTP RESPONSE         |
+-----------------------------+
| Status : 404 NOT FOUND      |
| Body   : Empty              |
+-----------------------------+
```

Simple definition for students

`ResponseEntity` is a Spring class that allows a REST controller to return the response body along with the HTTP status code and optional HTTP headers.